

# **DETAILED REPORT**

## **TASK #01: Titanic Survival Classification**

### **Table of Contents**

1. Introduction
2. Project Objective
3. Problem Statement
4. Dataset Description
5. Tools and Technologies
6. Project Structure
7. Methodology
8. Data Loading and Exploration
9. Data Preprocessing
10. Exploratory Data Analysis
11. Feature Selection
12. Train-Test Split
13. Machine Learning Model
14. Model Training
15. Model Prediction
16. Model Evaluation
17. Results and Discussion
18. Code Implementation
19. Screenshots and Visual Evidence
20. Challenges and Solutions
21. Learning Outcomes
22. Conclusion
23. Future Improvements

# 1. Introduction

Machine Learning is a branch of Artificial Intelligence that allows computers to learn patterns from data and make predictions without being explicitly programmed for every individual case.

For this project, the **Titanic dataset** from Kaggle was used to develop a binary classification model. The purpose of the project is to predict whether a passenger survived or did not survive based on passenger information such as passenger class, gender, and age, fare, and embarkation port.

The project covers the basic Machine Learning workflow, including data loading, data exploration, data cleaning, preprocessing, feature selection, model training, prediction, and evaluation.

## 2. Project Objective

The main objective of this project is to develop a Machine Learning classification model that can predict the survival status of Titanic passengers.

The project aims to:

- Load and understand the Titanic dataset.
- Explore the available passenger information.
- Identify and handle missing values.
- Select relevant features for prediction.
- Convert categorical data into numerical form.
- Divide the dataset into training and testing sets.
- Train a Logistic Regression classification model.
- Generate survival predictions.
- Evaluate the performance of the trained model.

## 3. Problem Statement

The Titanic dataset contains information about passengers who travelled on the RMS Titanic.

The task is to use passenger information to predict the value of the `Survived` variable.

The target has two possible values:

| Value | Meaning                   |
|-------|---------------------------|
| 0     | Passenger did not survive |
| 1     | Passenger survived        |

Therefore, this project is a **Binary Classification** problem.

## Binary Classification

Binary Classification is a Machine Learning task where the model predicts one of two possible classes.

In this project:

0 → Did not survive

1 → Survived

## 4. Dataset Description

The dataset used in this project is the **Titanic: Machine Learning from Disaster** dataset available on Kaggle.

The dataset contains information about Titanic passengers and their survival status.

The main columns include:

| Column             | Description                                   |
|--------------------|-----------------------------------------------|
| <b>PassengerId</b> | Unique identification number of the passenger |
| <b>Survived</b>    | Survival status of the passenger              |
| <b>Pclass</b>      | Passenger ticket class                        |
| <b>Name</b>        | Passenger name                                |
| <b>Sex</b>         | Passenger gender                              |
| <b>Age</b>         | Passenger age                                 |
| <b>SibSp</b>       | Number of siblings or spouses aboard          |
| <b>Parch</b>       | Number of parents or children aboard          |
| <b>Ticket</b>      | Ticket number                                 |
| <b>Fare</b>        | Passenger ticket fare                         |
| <b>Cabin</b>       | Cabin number                                  |
| <b>Embarked</b>    | Port where passenger boarded                  |

For this project, the following features were selected:

Pclass

Sex

Age

Fare

Embarked

The target variable was:

Survived

## **Dataset Source**

Kaggle Titanic Competition:

<https://www.kaggle.com/competitions/titanic>

---

# **5. Tools and Technologies**

The following tools and technologies were used:

## **Python**

Python was used as the primary programming language for data processing and Machine Learning.

## **Pandas**

Pandas was used for loading, manipulating, and analyzing the dataset.

## **NumPy**

NumPy provides numerical operations and array-based calculations.

## **Matplotlib**

Matplotlib was used to create data visualizations.

## Seaborn

Seaborn was used to create statistical visualizations.

## Scikit-learn

Scikit-learn was used for:

- Train-test splitting
- Logistic Regression
- Model training
- Predictions
- Model evaluation

## Visual Studio Code

Visual Studio Code was used as the development environment for writing and running Python code.

## Jupyter Notebook

Jupyter Notebook was used for exploratory data analysis, experimentation, visualizations, and documenting the Machine Learning workflow.

---

# 6. Project Structure

The project was organized using the following structure:

```
task_1_titanic_survival_classification/
```

```
|
```

```
|— data/
```

```
|   └─ raw/
```

```
|       └─ train.csv
```

```
|
```

```
|— notebooks/
```

```
|   └─ titanic_analysis.ipynb
```

```

|
|— src/
|   └─ titanic_classification.py
|
|— results/
|
|— README.md
└─ requirements.txt

```

## Explanation

- `data/raw/` contains the original dataset.
- `notebooks/` contains the exploratory Jupyter Notebook.
- `src/` contains the final Python implementation.
- `results/` contains project results and outputs.
- `README.md` contains project documentation.
- `requirements.txt` contains the required Python libraries.

| Name         | Date modified      | Type                | Size |
|--------------|--------------------|---------------------|------|
| .git         | 9/26/2026 12:30 AM | File folder         |      |
| data         | 9/15/2026 12:52 PM | File folder         |      |
| notebooks    | 9/25/2026 10:50 PM | File folder         |      |
| results      | 9/15/2026 12:52 PM | File folder         |      |
| src          | 9/25/2026 10:55 PM | File folder         |      |
| README       | 9/26/2026 12:20 AM | Markdown Source ... | 3 KB |
| requirements | 9/26/2026 12:22 AM | Text Document       | 2 KB |

**Figure 1:** Shows the organized folder structure of the Titanic Survival Classification project.

## 7. Methodology

The project follows a standard Machine Learning workflow:

Dataset

↓

Data Loading

↓

Data Exploration



Data Cleaning



Feature Selection



Data Preprocessing



Train-Test Split



Model Training



Prediction



Model Evaluation

This workflow allows the raw dataset to be transformed into a format that can be used by a Machine Learning algorithm.

## 8. Data Loading and Exploration

The Titanic dataset was loaded using the Pandas library.

```
df = pd.read_csv("data/raw/train.csv")
```

The `read_csv()` function reads the CSV file and stores the data in a Pandas DataFrame.

### Dataset Preview

The first five rows were displayed using:

```
df.head()
```

This helped in understanding the structure and values of the dataset.

```
import pandas as pd
df = pd.read_csv("D:/AI Engineering/Arc Technologies Internship/Data Science/titanic_survival_classification/data/raw/train.csv")
print(df.head())
```

[2] ✓ 0.0s Python

|   | PassengerId | Survived | Pclass | \ |
|---|-------------|----------|--------|---|
| 0 | 1           | 0        | 3      |   |
| 1 | 2           | 1        | 1      |   |
| 2 | 3           | 1        | 3      |   |
| 3 | 4           | 1        | 1      |   |
| 4 | 5           | 0        | 3      |   |

**Figure 2:** Shows the first five rows of the Titanic dataset after loading it into a Pandas DataFrame.

## 9. Dataset Information

The `info()` function was used to inspect the dataset.

```
df.info()
```

This provides information about:

- Column names
- Number of non-null values
- Data types
- Dataset structure

```
import pandas as pd
df = pd.read_csv("D:/AI Engineering/Arc Technologies Internship/Data Science/titanic_survival_classification/data/raw/train.csv")
print(df.info())
```

[4] ✓ 0.0s Python

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   PassengerId  891 non-null    int64  
1   Survived     891 non-null    int64  
2   Pclass       891 non-null    int64  
3   Name         891 non-null    str     
4   Sex          891 non-null    str     
5   Age          714 non-null    float64 
6   SibSp        891 non-null    int64  
7   Parch        891 non-null    int64  
8   Ticket       891 non-null    str     
9   Fare         891 non-null    float64 
10  Cabin        204 non-null    str     
11  Embarked     889 non-null    str     
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
None
```

**Figure 3:** Shows the structure, data types, and non-null information of the Titanic dataset.

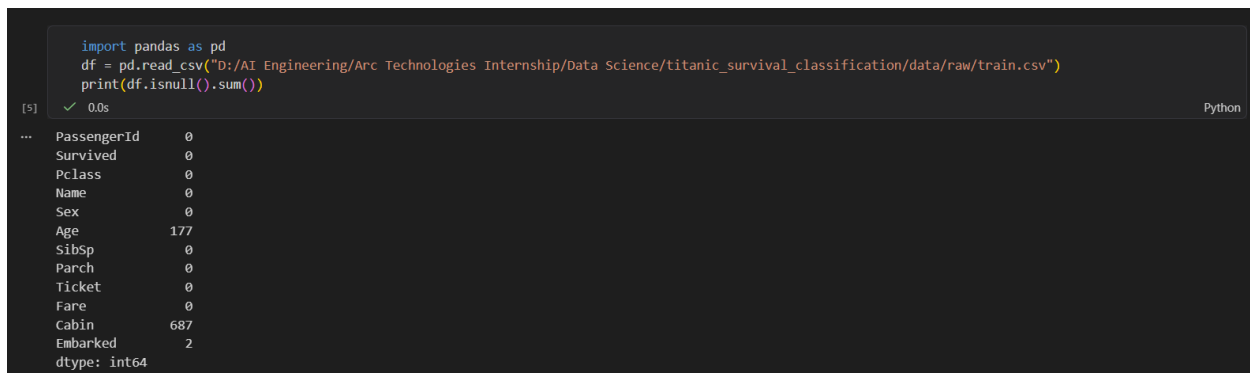
## 10. Missing Value Analysis

Missing values were checked using:

```
df.isnull().sum()
```

Missing values are important because Machine Learning models generally require properly prepared input data.

The analysis identified missing values in columns such as Age and Embarked.



```
import pandas as pd
df = pd.read_csv("D:/AI Engineering/Arc Technologies Internship/Data Science/titanic_survival_classification/data/raw/train.csv")
print(df.isnull().sum())
```

[5] ✓ 0.0s Python

|     |             |       |
|-----|-------------|-------|
| ... | PassengerId | 0     |
|     | Survived    | 0     |
|     | Pclass      | 0     |
|     | Name        | 0     |
|     | Sex         | 0     |
|     | Age         | 177   |
|     | SibSp       | 0     |
|     | Parch       | 0     |
|     | Ticket      | 0     |
|     | Fare        | 0     |
|     | Cabin       | 687   |
|     | Embarked    | 2     |
|     | dtype:      | int64 |

**Figure 4:** Shows the number of missing values present in each column before data preprocessing.

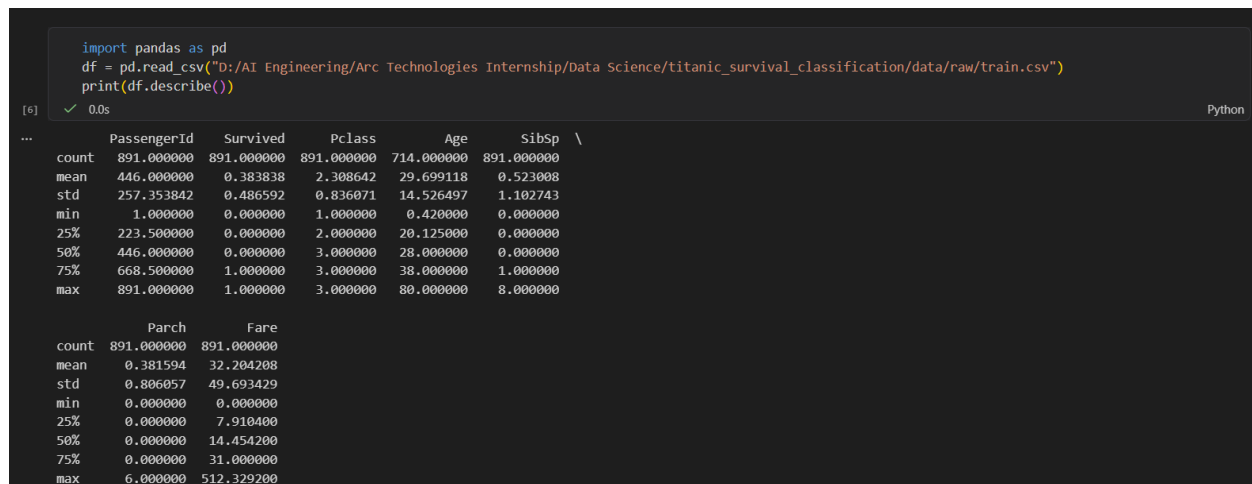
## 11. Statistical Analysis

The `describe()` function was used to obtain statistical information about numerical columns.

```
df.describe()
```

It provides information such as:

- Count
- Mean
- Standard deviation
- Minimum
- Maximum
- Quartiles



**Figure 5:** Shows the statistical summary of the numerical variables in the Titanic dataset.

## 12. Exploratory Data Analysis

Exploratory Data Analysis, commonly called **EDA**, is the process of examining and visualizing data to understand its patterns and characteristics.

Several visualizations were created to understand passenger survival.

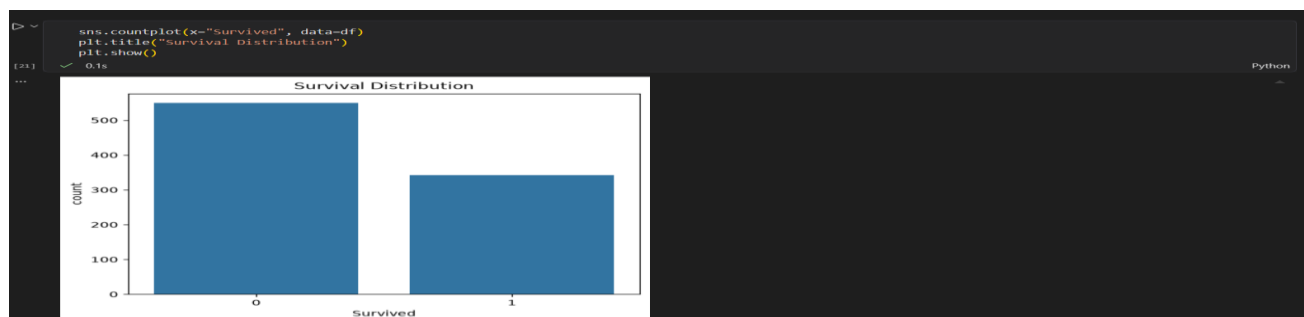
### 12.1 Survival Distribution

The distribution of survived and non-survived passengers was visualized.

```
sns.countplot(x="Survived", data=df)
```

```
plt.title("Survival Distribution")
```

```
plt.show()
```



**Figure 6:** Shows the distribution of passengers according to their survival status.

## 13. Survival by Gender

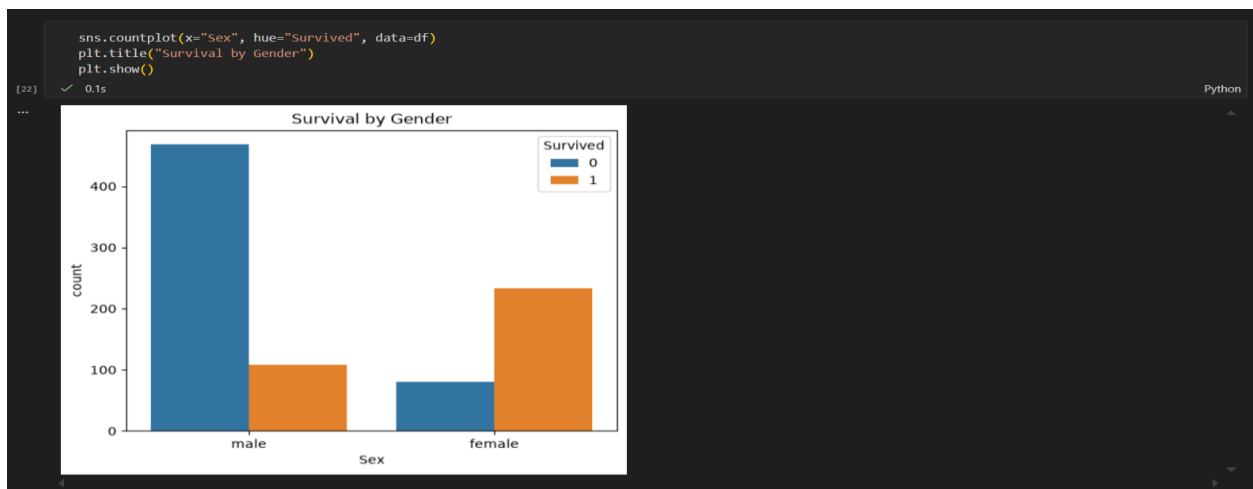
The relationship between gender and survival was explored.

```
sns.countplot(x="Sex", hue="Survived", data=df)

plt.title("Survival by Gender")

plt.show()
```

This visualization helps compare survival outcomes across the gender categories.



**Figure 7:** Shows the relationship between passenger gender and survival status.

## 14. Survival by Passenger Class

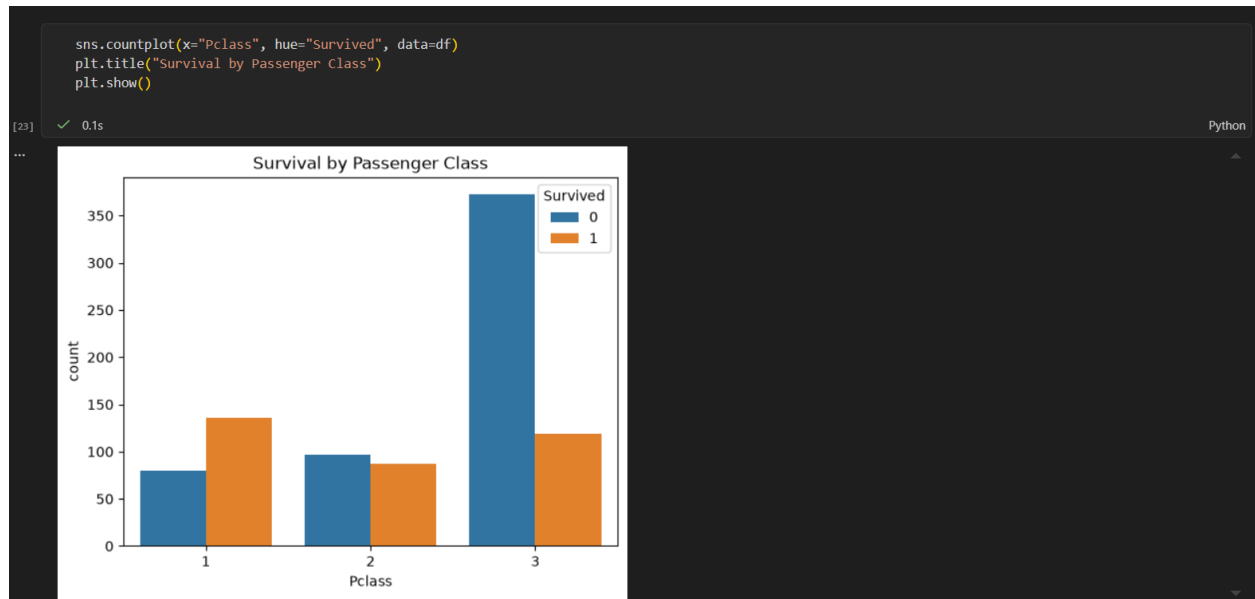
Passenger class was also analyzed.

```
sns.countplot(x="Pclass", hue="Survived", data=df)

plt.title("Survival by Passenger Class")

plt.show()
```

This allows the distribution of survival outcomes across different passenger classes to be visualized.



**Figure 8:** Shows the distribution of survival outcomes across different passenger ticket classes.

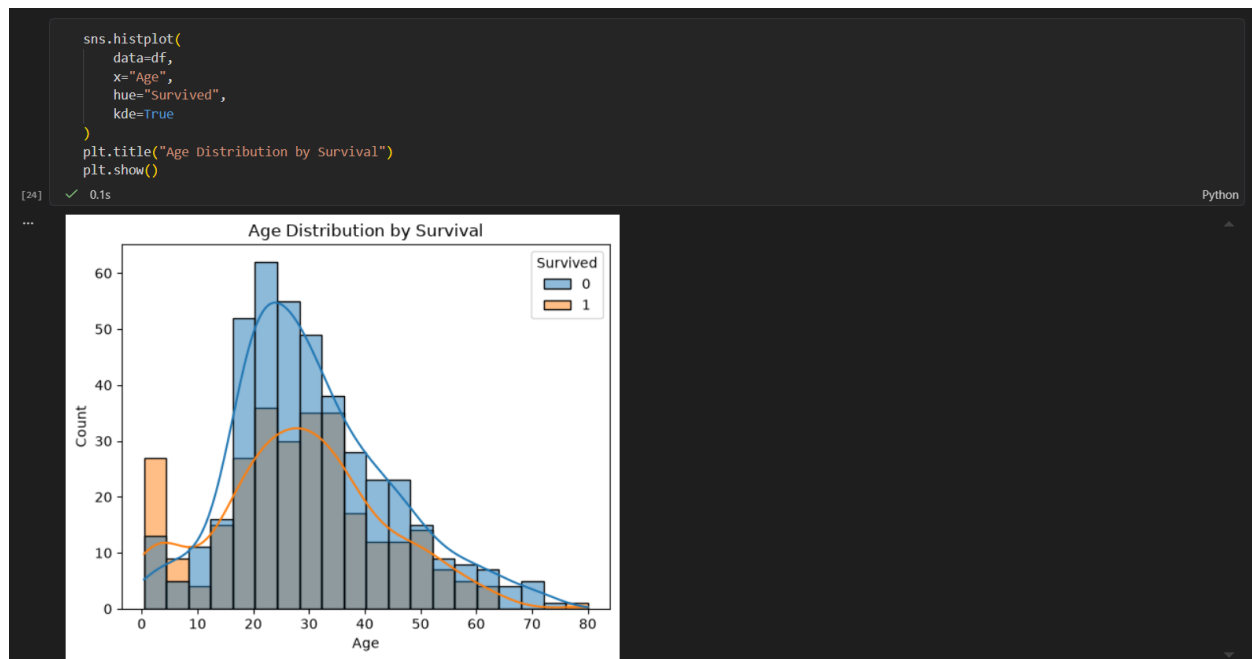
## 15. Age Distribution

The age distribution of passengers was visualized according to survival status.

```
sns.histplot(
    data=df,
    x="Age",
    hue="Survived",
    kde=True
)

plt.title("Age Distribution by Survival")

plt.show()
```



**Figure 9:** Shows the age distribution of Titanic passengers according to their survival status.

## 16. Feature Selection

The following features were selected:

```
features = ["Pclass", "Sex", "Age", "Fare", "Embarked"]
```

```
X = df[features]
```

```
y = df["Survived"]
```

### Features

Features are the input variables used by a Machine Learning model to make predictions.

In this project:

Pclass

Sex

Age

Fare

Embarked

are the features.

## **Target**

The target is the variable that the Machine Learning model attempts to predict.

Here:

Survived

is the target.

# **17. Data Preprocessing**

Data preprocessing was performed before training the Machine Learning model.

## **17.1 Handling Missing Age Values**

The missing `Age` values were replaced with the median age.

```
X["Age"] = X["Age"].fillna(X["Age"].median())
```

### **Explanation**

The median represents the middle value of the sorted data and is useful for replacing missing numerical values.

## **17.2 Handling Missing Embarked Values**

Missing `Embarked` values were replaced using the mode.

```
X["Embarked"] = X["Embarked"].fillna(  
    X["Embarked"].mode()[0]  
)
```

## Explanation

The mode is the value that occurs most frequently in a dataset.

## 17.3 Encoding Gender

Machine Learning models require numerical input, so gender was converted into numerical values.

```
X["Sex"] = X["Sex"].map({  
    "male": 0,  
    "female": 1  
})
```

The conversion was:

male → 0

female → 1

## 17.4 One-Hot Encoding

The `Embarked` column contains categorical values, so one-hot encoding was applied.

```
X = pd.get_dummies(  
    X,  
    columns=["Embarked"],  
    drop_first=True  
)
```

# 18. Train-Test Split

The prepared dataset was divided into training and testing datasets.

```
x_train, x_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

The dataset was divided into:

80% → Training data

20% → Testing data

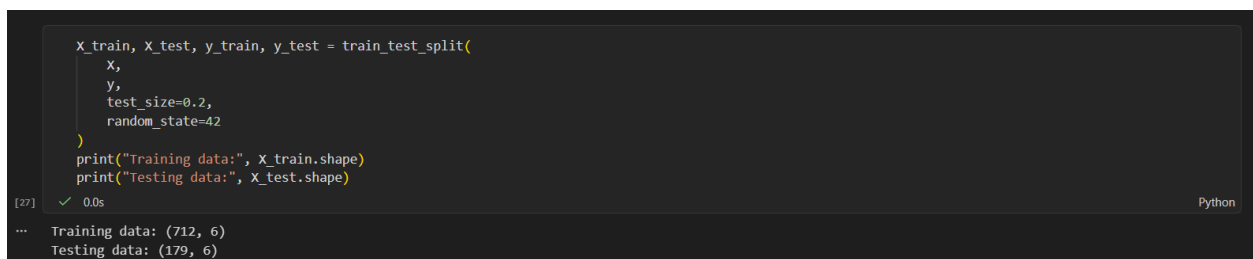
## Training Data

Training data is used by the model to learn patterns.

## Testing Data

Testing data is unseen data used to evaluate the trained model.

The `random_state=42` value ensures that the same split can be reproduced when the code is run again.



```
x_train, x_test, y_train, y_test = train_test_split(  
    x,  
    y,  
    test_size=0.2,  
    random_state=42  
)  
print("Training data:", x_train.shape)  
print("Testing data:", x_test.shape)
```

[27] ✓ 0.0s Python

... Training data: (712, 6)  
Testing data: (179, 6)

**Figure 11:** Shows the division of the dataset into training and testing subsets.

# 19. Machine Learning Model

## Logistic Regression

Logistic Regression was selected as the classification algorithm.

Despite the word "Regression" in its name, Logistic Regression is commonly used for classification problems.

### Definition

Logistic Regression is a Machine Learning classification algorithm used to estimate the probability of a class.

Since the Titanic problem has two possible outcomes:

0 → Did not survive

1 → Survived

it is suitable for binary classification.

# 20. Model Training

The Logistic Regression model was created using Scikit-learn.

```
model = LogisticRegression(max_iter=1000)
```

The model was then trained:

```
model.fit(X_train, y_train)
```

### Explanation

The `fit()` method trains the model using the training features and their corresponding target values.

During training, the model learns patterns between passenger characteristics and survival outcomes.

## 21. Making Predictions

After training, predictions were generated using the test data.

```
y_pred = model.predict(X_test)
```

The model uses the patterns learned during training to predict whether passengers in the test dataset survived.

### Definition

`predict()` uses a trained Machine Learning model to generate predictions for new data.

## 22. Model Evaluation

The model was evaluated using several performance metrics.

### 22.1 Accuracy

```
accuracy = accuracy_score(y_test, y_pred)

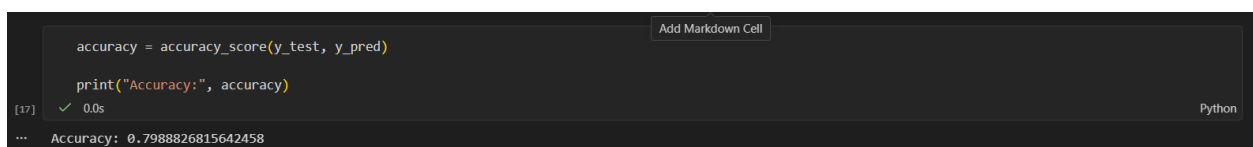
print("Accuracy:", accuracy)
```

### Definition

Accuracy is the proportion of total predictions that the model classified correctly.

The actual accuracy obtained from your run should be inserted here:

**Model Accuracy: [INSERT YOUR ACTUAL ACCURACY]%**

A screenshot of a Jupyter Notebook cell. The cell contains two lines of Python code: `accuracy = accuracy_score(y_test, y_pred)` and `print("Accuracy:", accuracy)`. Below the code, the output is displayed: `Accuracy: 0.798826815642458`. The cell is labeled with `[17]` and `Python` in the bottom right corner. There is a small button labeled "Add Markdown Cell" in the top right corner of the cell's background.

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

[17] ✓ 0.0s Python

... Accuracy: 0.798826815642458

**Figure 12:** Shows the accuracy achieved by the Logistic Regression model on the testing dataset.

## 23. Classification Report

The classification report was generated using:

```
print(classification_report(y_test, y_pred))
```

The report contains:

- Precision
- Recall
- F1-score
- Support

### Precision

Precision measures how many of the samples predicted as a particular class actually belonged to that class.

### Recall

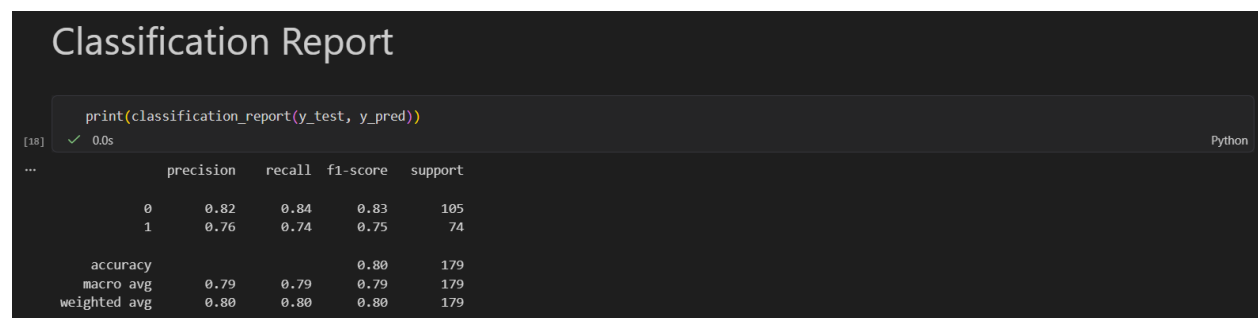
Recall measures how many of the actual samples belonging to a class were correctly identified.

### F1-score

F1-score combines precision and recall into a single metric.

### Support

Support represents the number of actual samples belonging to each class.



**Figure 13:** Shows the classification report containing precision, recall, F1-score, and support for the Logistic Regression model.

## 24. Confusion Matrix

A confusion matrix was generated to understand correct and incorrect predictions.

```
cm = confusion_matrix(y_test, y_pred)

print(cm)
```

A visual heatmap was also created:

```
sns.heatmap(

    cm,

    annot=True,

    fmt="d"

)

plt.xlabel("Predicted")

plt.ylabel("Actual")

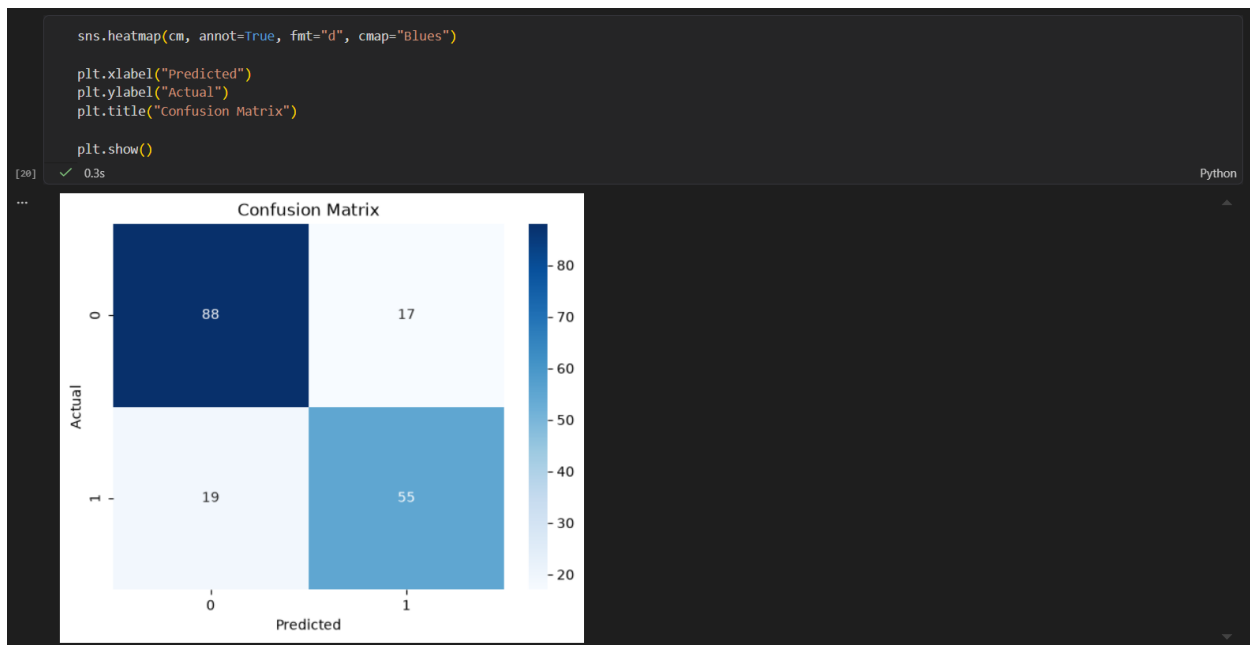
plt.title("Confusion Matrix")

plt.show()
```

### Definition

A confusion matrix is a table that shows the number of correct and incorrect predictions made by a classification model.

It helps identify how many passengers were correctly or incorrectly classified as survivors or non-survivors.



**Figure 14:** Shows the confusion matrix of the Logistic Regression model, displaying the correct and incorrect survival predictions.

## 25. Results and Discussion

After completing the preprocessing and model training process, the Logistic Regression model was evaluated using the test dataset.

The classification report provided precision, recall, and F1-score values for both survival classes.

The confusion matrix was also used to visualize the correct and incorrect predictions made by the model.

The results demonstrate that passenger information such as passenger class, gender, age, fare, and embarkation port can be used to build a Machine Learning model for predicting Titanic passenger survival.

## 26. Code Implementation

This section explains the main implementation used in the project.

### 26.1 Importing Libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (

    accuracy_score,

    classification_report,

    confusion_matrix

)
```

**Explanation:**

Pandas is used for data manipulation, while Scikit-learn provides the tools required for splitting the dataset, training the Logistic Regression model, and evaluating its performance.

### 26.2 Loading the Dataset

```
df = pd.read_csv("data/raw/train.csv")
```

**Explanation:**

The Kaggle Titanic training dataset is loaded into a Pandas DataFrame.

### 26.3 Selecting Features and Target

```
features = ["Pclass", "Sex", "Age", "Fare", "Embarked"]

X = df[features]

y = df["Survived"]
```

**Explanation:**

The selected passenger characteristics are stored in `x`, while the survival status is stored in `y`.

## 26.4 Handling Missing Values

```
X["Age"] = X["Age"].fillna(X["Age"].median())

X["Embarked"] = X["Embarked"].fillna(

    X["Embarked"].mode()[0]

)
```

**Explanation:**

Missing numerical values in `Age` are replaced using the median, while missing categorical values in `Embarked` are replaced using the mode.

## 26.5 Encoding Categorical Features

```
X["Sex"] = X["Sex"].map({

    "male": 0,

    "female": 1

})

X = pd.get_dummies(

    X,

    columns=["Embarked"],

    drop_first=True

)
```

**Explanation:**

Categorical variables are converted into numerical representations so they can be used by the Machine Learning model.

## 26.6 Splitting the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

### Explanation:

The dataset is divided into training and testing subsets using an 80:20 ratio.

---

## 26.7 Training the Model

```
model = LogisticRegression(max_iter=1000)  
  
model.fit(X_train, y_train)
```

### Explanation:

A Logistic Regression model is created and trained using the training dataset.

## 26.8 Generating Predictions

```
y_pred = model.predict(X_test)
```

### Explanation:

The trained model predicts the survival status of passengers in the testing dataset.

## 26.9 Evaluating the Model

```
accuracy = accuracy_score(y_test, y_pred)  
  
print("Accuracy:", accuracy)
```

```
print(classification_report(y_test, y_pred))  
  
print(confusion_matrix(y_test, y_pred))
```

**Explanation:**

The model is evaluated using accuracy, classification metrics, and a confusion matrix.

## 27. Challenges and Solutions

During the development of the project, several common Data Science challenges were addressed.

### Challenge 1: Missing Data

Some passenger records contained missing values.

**Solution:**

Missing numerical values were handled using the median, while missing categorical values were handled using the mode.

### Challenge 2: Categorical Data

The `Sex` and `Embarked` columns contained categorical values.

**Solution:**

Categorical values were converted into numerical representations using mapping and one-hot encoding.

### Challenge 3: Model Evaluation

Simply training a model does not show how well it performs on unseen data.

**Solution:**

The dataset was divided into training and testing subsets, and the model was evaluated using accuracy, classification report, and confusion matrix.

## 29. Learning Outcomes

This project provided practical experience with several fundamental Data Science and Machine Learning concepts.

Through this project, I learned how to:

- Work with a real-world dataset.
- Load and inspect data using Pandas.
- Identify missing values.
- Clean and preprocess data.
- Handle numerical and categorical variables.
- Perform exploratory data analysis.
- Create data visualizations.
- Select features and target variables.
- Split data into training and testing sets.
- Train a classification model.
- Generate predictions.
- Evaluate Machine Learning performance.
- Interpret a classification report.
- Understand a confusion matrix.
- Organize a Machine Learning project using a structured folder system.

## 30. Conclusion

The Titanic Survival Classification project provided practical experience in developing a complete Machine Learning classification workflow.

The project started with loading and exploring the Titanic dataset, followed by identifying and handling missing values. Relevant features were selected and categorical variables were converted into numerical representations.

The prepared dataset was divided into training and testing subsets, and a Logistic Regression model was trained to predict passenger survival. The model was then evaluated using accuracy, precision, recall, F1-score, and a confusion matrix.

Overall, the project helped build a practical understanding of the fundamental steps involved in a Machine Learning classification problem, from raw data preprocessing to model evaluation.

## 31. Future Improvements

The project can be further improved by:

1. Comparing Logistic Regression with other classification algorithms.
2. Testing Decision Tree and Random Forest models.

3. Performing feature engineering.
4. Applying hyper parameter tuning.
5. Comparing different preprocessing approaches.
6. Evaluating multiple models using the same metrics.
7. Generating predictions for the Kaggle test dataset.
8. Creating a Kaggle submission file.
9. Improving the model through systematic experimentation.

## 32. References

### Dataset

Kaggle — Titanic: Machine Learning from Disaster

<https://www.kaggle.com/competitions/titanic>

### Libraries

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn